PASSAGEM DE CONTEXTO

Tech Challenge Fase 4

Relatório de continuidade e situação do projeto

Referência: 8 de novembro de 2026

Arquitetura, branches, testes, riscos, pendências e roteiro de retomada.

Escopo: informação técnica de trabalho. A entrega acadêmica, o ambiente implantado e a integração real de pagamento ainda exigem validação e evidências.

Referência: 8 de novembro de 2026. Documento de passagem de contexto e acompanhamento técnico; não é o PDF final de submissão acadêmica nem comprova implantação em produção.

1. Leitura executiva

A solução foi organizada em três serviços: OS, Billing e Execution, cada qual em repositório próprio e com dados sob sua responsabilidade. A Saga é orquestrada pelo OS, com comandos e eventos assíncronos. Auth, plataforma AKS e infraestrutura PostgreSQL permanecem em repositórios complementares.

Há implementação e testes locais relevantes, incluindo execução dos três workers Nest como processos independentes, PostgreSQL e Cosmos emulados, broker Service Bus emulado, APIs HTTP e provedor de pagamento simulado. A última regressão distribuída registrada passou com 24 cenários e 72 passos. Isso não demonstra, por si só, deploy Kubernetes real, comunicação na nuvem, compra de teste no Mercado Pago, isolamento das identidades reais ou observabilidade no ambiente implantado.

A entrega acadêmica permanece incompleta: é necessário concluir aceite de ambiente, evidências por SHA, vídeo e PDF de submissão com os dados reais dos integrantes. Não existe percentual atual confiável de conclusão: a avaliação antiga de 54% foi feita em outro momento e não deve ser apresentada como medição atual.

2. O que o enunciado exige

O enunciado original exige no mínimo três microsserviços independentes, cada um com repositório, infraestrutura e banco próprios; pelo menos um banco SQL e um NoSQL; REST quando necessário, mensageria assíncrona e ausência de acesso direto ao banco alheio. Exige Saga justificada com compensação, integração Mercado Pago, testes unitários, BDD de fluxo completo, cobertura mínima de 80% por serviço e análise de qualidade por SonarQube ou similar.

Também exige CI/CD independente, build, testes, qualidade, deploy automatizado em Kubernetes, proteção da main, reaproveitamento de observabilidade, documentação REST e artefatos de cada serviço. O vídeo público ou não listado no YouTube/Vimeo deve ter até 15 minutos e mostrar jornada, falha/Saga, deploy com testes e rastreamento. O PDF do portal deve conter participantes e identificações, links dos repositórios e vídeo, diagrama final, estratégia de Saga e justificativas.

As tecnologias específicas escolhidas - Service Bus Standard, Cosmos, PostgreSQL, NestJS e orquestração no OS - são decisões do projeto, não exigências literais do enunciado. O plano complementar acrescenta contratos, deadlines, regras de segurança e critérios de validação.

3. Estado verificado dos repositórios

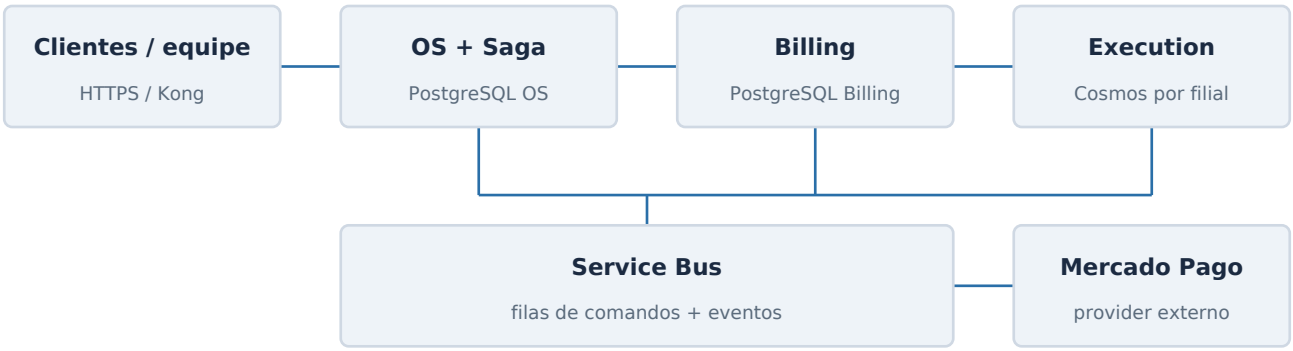
As branches abaixo foram conferidas no remoto; os commits são o recorte de referência desta passagem, não garantem que a branch continuará imóvel. As branches de trabalho foram renomeadas para o padrão do projeto (imp/ e docs/); a renomeação fechou os PRs antigos, substituídos pelos PRs listados, abertos para revisão e ainda não mesclados em main ou development.

- OS: soat-api, branch imp/fase4-os-service, commit 4d1ddcb; PR 15. Inclui deploy kind no CI e README da Fase 4.
- Billing: soat-billing-service, branch imp/fase4-billing-service, commit abf5fb5; PR 2. Inclui deploy kind, overrides de dependências e README da Fase 4.
- Execution: soat-execution-service, branch imp/fase4-execution-service, commit 0cf7b65; PR 2. Inclui deploy kind e README da Fase 4.

- Auth Function: soat-auth-function, branch imp/fase4-auth-http, commit a2ecea4; PR 25. CI de qualidade passou.
- AKS: soat-aks-infra, branch imp/fase4-plataforma, commit 13afb2e; PR 26. O check obrigatório validate passou; os jobs de plano falham no acesso ao state com AccountIsDisabled.
- PostgreSQL: soat-postgres-infra, branch imp/fase4-bancos-servicos, commit 34015e7; PR 8. O check validate passou; os planos também falham com AccountIsDisabled.
- Documentos centrais: FIAP_TechChallenges / docs/fase4, branch docs/fase4-documentacao. A branch contém plano, roteiro inicial, auditoria, progresso, revisão, base acadêmica e guia de retomada.
- Dependências de Billing: os overrides da antiga branch experimental imp/fase4-dependencias-billing (commit 6d86a87) foram incorporados à branch principal de Billing.

Para clonar em outro computador: git clone -b NOME_DA_BRANCH URL_DO_REPOSITORIO. Em um clone existente, preserve as alterações locais, execute git fetch origin, selecione a branch correspondente e use git pull --ff-only. O guia remoto retomada-outro-computador.md relaciona as branches. Credenciais e o PDF original do desafio não foram incluídos nos repositórios.

4. Arquitetura e propriedade dos dados



Auth Function consulta OS via HTTPS. O desenho representa a arquitetura pretendida, sujeita ao aceite real.

OS mantém clientes, veículos, abertura e acompanhamento, histórico, autenticação administrativa, autorização, relatórios e estado da Saga. Billing é a autoridade de orçamento versionado, itens e preços aprovados, cobrança, pagamento, webhook, reembolso e notificações. Execution responde por catálogo, peças, serviços, estoque por filial, diagnóstico, reserva, fila e reparo. Auth consulta o OS por HTTPS para validar o cliente, usando credencial própria de integração.

O desenho previsto usa PostgreSQL separado para OS e Billing, podendo compartilhar o servidor físico sem compartilhar bancos ou credenciais, e Cosmos DB for NoSQL para Execution, com partição por filial. A mensageria usa duas filas de comandos e um topic de eventos, com assinatura e verificação dos produtores. Cada serviço deve ter identidade, secrets, build, imagem e deploy próprios; cluster e namespace podem ser compartilhados.

As estruturas e adapters foram implementados e exercitados localmente, mas banco, rede, permissões, identidade, Service Bus e Cosmos reais ainda precisam ser confirmados. Não se deve inferir isolamento real apenas pelos manifestos ou pelos emuladores.

5. Jornada e garantias da Saga

A jornada planejada é: abrir OS; diagnosticar e fixar itens e preços de referência; gerar e enviar orçamento; obter aprovação vinculada ao cliente e à versão; reservar peças; criar cobrança; confirmar pagamento; enfileirar; iniciar e concluir o reparo; finalizar a Saga. A entrega do veículo é posterior e exige situação operacional e financeira apropriada.

O OS persiste estado, operações, histórico, inbox e outbox; os serviços registram seus efeitos e mensagens locais de forma durável. A entrega é pelo menos uma vez, com messageId e operationId estáveis, deduplicação, versões de agregado locais e correlação/causalidade. Fatos fora de ordem são preservados e reaplicados, em vez de depender de ordenação global.

O cancelamento do trabalho precisa comprovar que ele não começou. Pagamento confirmado exige reembolso adequado antes da liberação da reserva. Se não houver pagamento e uma cobrança criada precisar ser cancelada, o Billing registra intenção, consulta o estado canônico, envia uma única tentativa e reconcilia incertezas sem reenviar a mutação. O OS só encaminha a liberação após confirmação e ausência de conflitos. Os recursos desse caminho estão protegidos por ENABLE_SAGA_CHARGE_CANCELLATION e ENABLE_MERCADO_PAGO_CANCEL_COMMANDS, desativados por padrão.

Timeout, falha de rede ou ausência de evento não são prova de cancelamento, estorno ou liberação. Evidência divergente produz auditoria e evento de bloqueio, deixando a Saga em reconciliação. Aviso tardio impede novas tentativas de publicação de liberação pendente; se uma publicação já tiver sido reclamada, o resultado permanece incerto e é necessário consultar Execution. Trabalho físico iniciado exige incidente operacional, não rollback fictício.

Defaults do plano: aprovação de orçamento em 24 horas, cobrança em 15 minutos, revisão da reserva em 20 minutos, verificação técnica a cada 60 segundos e até cinco tentativas transitórias. Esses valores são configurações iniciais, não evidência de configuração do ambiente real; o vencimento da reserva solicita verificação, não liberação automática.

6. Pagamento, segurança e recuperação

Billing contém adapter da Orders API para criar, consultar, cancelar e reembolsar; persistência de intenção, referência e chaves idempotentes; validação de assinatura do webhook; deduplicação de notificações; consulta canônica para validar identificação, valor, moeda e estado. Os testes automatizados usam provedor simulado. A prova real ainda requer aplicação, contas de teste, loja/caixa, endpoint HTTPS, callback e credenciais apropriadas.

Comandos e eventos são validados por contrato e por assinatura de produtor; autenticação de cliente e administrativa é distinta da credencial interna Auth-OS. Tokens, CPF, QR completo e dados financeiros não devem ser colocados em logs ou relatórios. Em HML é necessário verificar chaves, issuer/audience, Key Vault, ServiceAccounts, OIDC e negação explícita de acesso cruzado entre bancos.

Há ferramentas de recuperação para outbox e DLQ, com seleção por mensagem, validação, auditoria e ACK após persistência. Elas não substituem diagnóstico: uma mensagem pode já ter sido processada e uma operação financeira pode permanecer incerta. Consulte docs/operacao-fase4.md no OS e docs/reconciliacao-operacional.md no Billing. Algumas retomadas, especialmente evidência divergente, ainda não possuem comando automático seguro; não alterar status diretamente para contornar verificações.

7. Testes e evidências disponíveis

- A regressão completa registrada passou com 24 cenários e 72 passos pelo broker local. Incluiu jornada, compensação, workers Nest em três processos, HTTP, interrupções, reentrega, cancelamento antes do pagamento, timeout e evidência divergente. Mercado Pago permaneceu simulado; Cosmos e Service Bus foram emulados.
- Billing passou em 209 testes unitários na avaliação das dependências e 51 testes de integração no PostgreSQL local. A suíte OS registrou 294 testes unitários e 62 integrações de Saga; os relatórios e gates devem ser conferidos para os SHAs finais de entrega, não tratados como média entre serviços.

- O fluxo de cancelamento com os três workers foi testado com uma chamada ao provedor simulado; o cenário de resposta perdida confirmou reconciliação sem repetir a chamada. O aviso de evidência divergente percorreu o broker e manteve reserva e Saga em reconciliação.
- Houve falha anterior intermitente na integração de reembolso e um retorno de lock inválido do emulador após persistência. Repetições posteriores passaram; não foi demonstrada a causa nem provada relação entre os episódios.
- Uma repetição contaminada por bancos/filas reutilizados apresentou falhas e foi interrompida. A execução em bancos e broker limpos passou; não se deve generalizar isso como causa comprovada de toda intermitência histórica.
- O workflow manual do BDD fixa SHAs e depende de acesso entre repositórios; na auditoria registrada faltava CROSS_REPO_READ_TOKEN. Confirmar o estado atual do secret e executar o workflow para a combinação exata antes de usar seu resultado como prova remota.

8. CI/CD, qualidade e infraestrutura

OS, Billing e Execution possuem pipelines independentes, Dockerfiles, manifestos e verificações de build, testes e qualidade. Os workflows de publicação/deploy de Billing e Execution são protegidos por trava e vinculados ao SHA aprovado; o fato de o CI passar não prova publicação ou rollout real. SonarJS com regras selecionadas foi usado como gate similar; a equivalência ao SonarQube completo não foi comprovada e deve ser explicitada na revisão acadêmica.

Cada pipeline de serviço também possui o job Deploy Kubernetes (kind): cria um cluster kind efêmero no runner, carrega a imagem do SHA avaliado, sobe as dependências do serviço, executa as migrations e aplica o overlay kind, validando rollout, imagem e rotas. No OS, o teste inclui login administrativo e consulta da fila operacional no banco da Saga; em Billing, consulta autenticada ao PostgreSQL próprio; em Execution, o acesso ao Cosmos permanece fechado sem identidade Azure. A publicação HML só ocorre após esse job. É o deploy automatizado verificável enquanto a assinatura Azure estiver desativada; não substitui o ensaio no AKS. Os totais de cobertura de cada SHA são publicados no resumo do job e como anotação.

As últimas execuções Terraform de AKS e PostgreSQL consultadas falharam em 31/10/2026: o backend de state retornou HTTP 403 AccountIsDisabled. O responsável informou que a conta Azure foi desativada para evitar cobrança e não pode ser reativada agora. Não executar terraform apply, provisionamento, deploy ou reativação sem decisão expressa de ambiente e custo. Os states e o banco legado devem ser preservados.

Antes de qualquer cutover, revisar branches e PRs, regras de proteção, módulos e states, custos e quotas, credenciais e rede, e validar a combinação exata de imagens. Usar dados sintéticos no primeiro aceite, garantir escritor único e preservar rollback seguro. Migração de dados históricos, backup/restore e reconciliação de IDs precisam de plano separado se forem necessários.

9. Segurança de dependências e experimento separado

O audit da instalação local da branch principal de Billing registrou alertas altos na cadeia do CLI de migração Prisma e moderados em uma cadeia transitiva de metadados/HTTP. A imagem de runtime foi construída com o CLI Prisma removido; o prune registrou cinco alertas moderados. O audit de outras instalações locais também apontou alertas, devendo-se distinguir grafo instalado, runtime efetivo e imagem de migração.

Os overrides de deepmerge-ts 8.0.2 sob @prisma/config e gaxios 8.3.2 sob gcp-metadata foram incorporados à branch principal de Billing, com a verificação em loopback dos metadados no CI. Na instalação revisada, npm audit --omit=dev não apontou vulnerabilidades. Os overrides cruzam as faixas declaradas pelos consumidores e não têm validação em GCP real; devem ser removidos quando as dependências oficiais incorporarem as correções.

10. Como reproduzir localmente

Leia primeiro os READMEs e workflows de cada branch. Use Node 24 onde exigido e execute npm ci, geração de clientes, lint, build, testes unitários e testes de contrato de cada serviço. Os testes que exigem banco ou emulador devem receber URLs de teste locais e falhar claramente se faltarem requisitos; não aponte scripts de teste para bancos reais.

Para o aceite distribuído, prepare bancos de OS e Billing distintos e descartáveis, Cosmos e Service Bus emulados, configure OS_TEST_DATABASE_URL, BILLING_TEST_DATABASE_URL, COSMOS_TEST_ENDPOINT, SERVICE_BUS_TEST_CONNECTION_STRING e os caminhos absolutos BILLING_REPO_DIR e EXECUTION_REPO_DIR. A documentação e .github/workflows/bdd-distribuido.yml no OS contêm a sequência, migrações e manifestos dos emuladores. Execute npm run test:bdd-distributed no OS. Use somente fixtures e provider simulado; rode suites de broker em sequência e parta de filas e bancos de teste isolados.

Não copiar credenciais de produção para testes. A execução local depende dos recursos e licenças dos emuladores aceitos pelo responsável. Para provar a integração real é obrigatório um ensaio separado autorizado e identificado por versões.

11. Plano de retomada: ações locais sem setup externo

1. Revisar o diff e os PRs rascunho, especialmente a sinalização de bloqueio, corridas entre envio e liberação e procedimentos de reconciliação. Registrar os resultados por commit.
2. Acompanhar as versões oficiais de @prisma/config e gcp-metadata e remover os overrides incorporados quando possível. Não tratar override major como compatibilidade garantida somente pelo audit.
3. Revisar os alertas das demais branches e as imagens reais de runtime versus migração; manter gate e justificativas atualizados. Completar os procedimentos de rearmar incidentes que ainda não possuem ferramenta segura.
4. Revalidar Swagger/OpenAPI, schemas de mensagens, READMEs, diagramas e exemplos contra as versões finais; manter o quality gate e as exclusões de cobertura explícitos.
5. Fixar manifesto de SHAs/imagens, revisar matriz R01-R43 e backlog T01-T30, executar regressões locais e rastrear evidências. A execução remota do BDD precisa de acesso entre repositórios.

12. Pendências que dependem de decisões, contas ou ambiente

1. Decidir um ambiente viável sem cobrança indesejada e definir política de custo. A assinatura/conta Azure existente não deve ser reativada automaticamente.
2. Disponibilizar, no ambiente escolhido, state seguro, rede/HTTPS, quotas, Service Bus, bancos, Cosmos, identidades e secrets; verificar acesso cruzado negado.
3. Configurar contas de teste, aplicação, loja/caixa, webhook e credenciais Mercado Pago, e ensaiar criação, pagamento, cancelamento, reembolso e reconciliação sem transações reais.
4. Configurar SMTP e New Relic, ou alternativas aceitas, incluindo variáveis e acesso; comprovar o trace da mesma OS e métricas sem dados pessoais.
5. Configurar o secret CROSS_REPO_READ_TOKEN no OS (ausente na última consulta) para executar o BDD distribuído remoto, revisar e mesclar os PRs e garantir acesso dos avaliadores aos repositórios privados e evidências.

6 Executar deploy autorizado no Kubernetes, aceite completo, cenários de falha e recuperação, isolamento e demonstração do SHA aprovado. Produzir a prova de rollback e escritor único antes do cutover.

7 Informar nomes e identificadores dos participantes e prazo acadêmico; gravar vídeo de até 15 minutos, testar o acesso e preparar o PDF final do portal. Revisar e submeter ao portal.

Essas sete frentes agrupam tarefas correlatas, não são uma contagem de cliques nem significam que apenas sete ações são necessárias. Este relatório é um apoio à continuidade; não substitui o PDF de entrega com dados e evidências reais.

13. Referências para continuidade

- Enunciado original da Fase 4 - conferir o PDF fornecido pelo grupo; ele não foi publicado nesta branch.
- Plano completo e matriz R01-R43.
- Roteiro de início.
- Registro de progresso.
- Revisão histórica - a estimativa e a matriz refletem uma revisão anterior, não uma auditoria atual.
- Guia de retomada e base para entrega acadêmica.
- CI OS, CI Billing, CI Execution, CI Auth.
- Terraform AKS com falha e Terraform PostgreSQL com falha: AccountIsDisabled no backend consultado; as execuções são históricas e não substituem verificação atual.

14. Matriz de requisitos e backlog

A matriz seguinte transcreve os 43 requisitos e seus critérios de evidência do plano, seguida do backlog completo T01-T30. Ela é uma lista de verificação para revisão, não um atestado de que cada requisito já foi concluído. O estado efetivo deve ser vinculado a provas dos SHAs e do ambiente final; o relatório histórico oferece uma fotografia anterior que precisa ser atualizada.

Requisitos R01-R43

- R01 - Mínimo de três microsserviços independentes. Evidência prevista: OS, Billing e Execution executam separadamente.
- R02 - Repositório próprio por microsserviço. Evidência prevista: Três URLs de aplicação; infraestrutura não conta como serviço.
- R03 - Infraestrutura própria por microsserviço. Evidência prevista: Manifestos/IaC, identidade, deploy e banco de cada dono; cluster/namespace compartilhados.
- R04 - Banco próprio por microsserviço. Evidência prevista: Três bancos e credenciais isoladas.
- R05 - Pelo menos um banco SQL. Evidência prevista: PostgreSQL exclusivo de OS e de Billing.
- R06 - Pelo menos um banco NoSQL. Evidência prevista: Cosmos DB for NoSQL exclusivo de Execution.
- R07 - Nenhum acesso ao banco de outro serviço. Evidência prevista: Testes negativos de credenciais/RBAC; Function via API.
- R08 - Estratégia de REST síncrono quando necessário. Evidência prevista: OpenAPI, autenticação interna, timeout e decisões documentadas.
- R09 - Mensageria assíncrona e integração desacoplada. Evidência prevista: Broker real, consumidores e eventos funcionando.

- R10 - Saga do fluxo transacional da OS. Evidência prevista: Estado durável, comandos, respostas e retomada.
- R11 - Rollback/compensação em falhas de etapas. Evidência prevista: Matriz de compensação e testes de falha.
- R12 - Escolha de Saga documentada e justificada no README. Evidência prevista: ADR e README de cada participante.
- R13 - Integração com Mercado Pago para pagamentos. Evidência prevista: Order/QR e pagamento em ambiente de teste real, webhook e consulta.
- R14 - Testes unitários em todos os microsserviços. Evidência prevista: Suites locais de cada novo serviço.
- R15 - Pelo menos um fluxo completo com BDD. Evidência prevista: Feature executável com os três serviços e broker.
- R16 - Cobertura mínima de 80% por serviço. Evidência prevista: Gates separados e relatórios identificados por SHA.
- R17 - SonarQube ou similar no CI. Evidência prevista: Quality Gate bloqueante por serviço.
- R18 - CI/CD independente por microsserviço. Evidência prevista: Cada repo publica e implanta sua própria imagem.
- R19 - Build em cada pipeline. Evidência prevista: Job obrigatório por repo e SHA.
- R20 - Testes automatizados em cada pipeline. Evidência prevista: Unitários bloqueantes em cada CI; integração/BDD na validação protegida do conjunto.
- R21 - Verificação de qualidade em cada pipeline. Evidência prevista: Lint e Sonar/alternativa comprovada.
- R22 - Deploy automatizado em Kubernetes. Evidência prevista: Três deployments e demonstração de rollout.
- R23 - Main protegida, com PR obrigatório. Evidência prevista: Captura/export das regras exigindo PR, sem testar push contra código válido.
- R24 - Checagens automáticas exigidas para merge. Evidência prevista: Merge bloqueado com teste/quality gate reprovado.
- R25 - Reutilizar monitoramento/observabilidade da Fase 3. Evidência prevista: New Relic/OTel estendidos aos serviços e mensagens.
- R26 - Código-fonte por repo. Evidência prevista: Código completo, instruções e tag da entrega.
- R27 - Dockerfile por repo de microsserviço. Evidência prevista: Três imagens executáveis e reproduzíveis.
- R28 - Manifestos Kubernetes por repo. Evidência prevista: Bases/overlays e workers de cada dono.
- R29 - Pipelines versionados por repo. Evidência prevista: Workflows próprios e links de execução.
- R30 - Evidências de cobertura no README. Evidência prevista: Links/prints de cada serviço, sem média global.
- R31 - Documentação da arquitetura do serviço. Evidência prevista: Contexto, dados, contratos, Saga e operação.
- R32 - Swagger ou collection Postman atualizada. Evidência prevista: Documentação e exemplos de cada serviço.

- R33 - Vídeo YouTube/Vimeo, público ou não listado, até 15 min. Evidência prevista: URL acessível e duração verificada.
- R34 - Vídeo com fluxo completo da OS entre serviços. Evidência prevista: Trecho de abertura a conclusão.
- R35 - Vídeo com Saga e tratamento de falhas. Evidência prevista: Falha controlada, compensação e retomada.
- R36 - Vídeo com deploy automatizado e validação de testes. Evidência prevista: Pipeline de pelo menos um serviço efetivamente aplicado.
- R37 - Vídeo com monitoramento/rastreamento distribuído. Evidência prevista: Traces e métricas de uma mesma OS.
- R38 - PDF do portal com participantes e identificação. Evidência prevista: Nomes e identificadores acadêmicos conferidos.
- R39 - PDF com links dos repositórios. Evidência prevista: Links dos serviços e infraestrutura revisados.
- R40 - PDF com link do vídeo de até 15 minutos. Evidência prevista: Link sem pedido de acesso.
- R41 - PDF com diagrama geral: serviços, bancos, comunicação. Evidência prevista: Diagrama final coerente com o deploy.
- R42 - PDF descrevendo orquestração/coreografia escolhida. Evidência prevista: Estratégia e compensações explicadas.
- R43 - PDF justificando divisão e tecnologias. Evidência prevista: Justificativas ligadas ao domínio e à base existente.

Backlog T01-T30

- T01 - Fixar baseline e reconciliar branches. Prioridade: P0. Responsabilidade: Plataforma + todos os 4 atuais. Dependências: -. Critério: SHAs revalidados, correções de AKS preservadas, PRs revisados e baseline documentado; sem regressão/produção involuntária.
- T02 - Criar repos de serviços e proteção. Prioridade: P0. Responsabilidade: Grupo / GitHub. Dependências: T01. Critério: Billing/Execution criados; main com PR/checks obrigatórios nos 3; prova das regras que hoje não foi possível ler.
- T03 - Spike e IaC da mensageria. Prioridade: P0. Responsabilidade: Plataforma / AKS. Dependências: T01. Critério: Standard/TLS/identidade, topic, duas filas, subscription OS e DLQ validados; mensagem sobrevive à queda; sem sessions obrigatórias.
- T04 - ADRs e contratos v1. Prioridade: P0. Responsabilidade: Domínio + todos os serviços. Dependências: T01. Critério: Fronteiras, status, pagamento antes do reparo, Saga e envelopes aprovados em repo; OpenAPI e JSON Schemas validados, sem exigir AsyncAPI formal.
- T05 - Bancos SQL e privilégios. Prioridade: P0. Responsabilidade: Dados / PostgreSQL + OS/Billing. Dependências: T02,T04. Critério: DBs isolados, runtime/migration separados, grants restritos, backup e teste de acesso cruzado negado.
- T06 - Spike e banco NoSQL. Prioridade: P0. Responsabilidade: Execution + Plataforma. Dependências: T02,T04. Critério: Cosmos, partição/índices, batch+ETag+outbox/inbox testados; limites/custos e conectividade documentados.
- T07 - Desacoplar Auth do SQL alheio. Prioridade: P0. Responsabilidade: Auth + OS. Dependências: T04,T08,T17 (HTTPS). Critério: Adapter HTTP pelo Kong HTTPS, credencial exclusiva Function/OS, testes de CPF ativo/inativo/erro; sem DATABASE_URL/grants do banco OS.

- T08 - Extrair OS Service. Prioridade: P1. Responsabilidade: OS / soat-api. Dependências: T04,T05. Critério: Cliente/veículo/OS/histórico/relatórios/autorização próprios, filialId configurado; migrations novas; sem catálogo/baixa/cálculo financeiro local.
- T09 - Extrair Billing Service. Prioridade: P1. Responsabilidade: Billing. Dependências: T04,T05. Critério: Orçamento versionado, decisão do cliente, total em centavos, notificações duráveis e dados próprios; testes preservados/adaptados.
- T10 - Extrair Execution Service. Prioridade: P1. Responsabilidade: Execution. Dependências: T04,T06. Critério: Catálogo/estoque/diagnóstico/fila/reparo próprios e adapter NoSQL; isolamento por filial.
- T11 - Container e manifestos por serviço. Prioridade: P1. Responsabilidade: Cada serviço. Dependências: T02 + esqueleto do próprio serviço. Critério: Uma imagem/Deployment por serviço em HML, API+loops, probes/recursos/identidades e migration quando SQL; sem esperar a extração dos outros.
- T12 - Outbox/inbox e contratos. Prioridade: P0. Responsabilidade: Cada serviço. Dependências: T03 + banco/esqueleto do próprio serviço. Critério: Commit+outbox e inbox+efeito atômicos por serviço, em paralelo; IDs, retry/DLQ e reentrega testados.
- T13 - Orquestrador durável e timers. Prioridade: P1. Responsabilidade: OS. Dependências: T04,T08,T12. Critério: Estado/versão/leases/deadlines persistidos; reinício e concorrência sem duplicar próxima etapa.
- T14 - Integrar fluxo até orçamento. Prioridade: P1. Responsabilidade: OS + Billing + Execution. Dependências: T09,T10,T13. Critério: Abertura -> diagnóstico -> orçamento -> aprovação/recusa convergem por eventos, sem consulta de DB cruzada.
- T15 - Reserva, cancelamento e compensações iniciais. Prioridade: P0. Responsabilidade: Execution + OS + Billing. Dependências: T10,T12-T14. Critério: Estoque concorrente, expiração/recusa e liberação idempotentes; estados e ACKs persistidos.
- T16 - Fluxo de fila, execução e conclusão. Prioridade: P1. Responsabilidade: Execution + OS. Dependências: T14,T15,T19. Critério: Pagamento confirmado autoriza fila; consumo único, início/finalização; Saga conclui em FINALIZADA e entrega posterior permanece no OS.
- T17 - Spike Mercado Pago real de teste. Prioridade: P0. Responsabilidade: Billing + responsável pela conta. Dependências: T04. Critério: Aplicação/contas/loja/caixa/HTTPS e compra teste confirmados; API vigente e elegibilidade registradas, sem valor real.
- T18 - Cobrança e QR no adapter. Prioridade: P1. Responsabilidade: Billing. Dependências: T09,T12,T17. Critério: Criação idempotente com Orders API, QR, IDs persistidos, valor/ref/ambiente corretos; timeout reconciliável.
- T19 - Webhook e conciliação. Prioridade: P0. Responsabilidade: Billing. Dependências: T18. Critério: HMAC, inbox, consulta canônica, dedupe/ordenação e polling de pendências; pagamento confirmado uma única vez.
- T20 - Compensação financeira completa. Prioridade: P0. Responsabilidade: Billing + OS + Execution. Dependências: T15,T16,T19. Critério: Cancelamento/reembolso, pagamento tardio e falha pós-pagamento testados; pendências nunca escondidas como sucesso.
- T21 - Suites e cobertura por serviço. Prioridade: P1. Responsabilidade: Cada serviço. Dependências: T08-T10, incremental. Critério: Unitários/integração com >=80% no escopo justificado; adapter/consumidor críticos cobertos; relatórios por SHA.
- T22 - BDD do fluxo distribuído. Prioridade: P1. Responsabilidade: Qualidade + todos. Dependências: T16,T20,T21. Critério: Gherkin executável com 3 serviços/bancos/broker; fluxo feliz e uma compensação completos.

- T23 - Matriz de resiliência e concorrência. Prioridade: P1. Responsabilidade: Qualidade + todos. Dependências: T12,T13,T20. Critério: Casos da seção 11.3 aprovados, inclusive queda/reentrega/refund/corridas; evidências identificam versões.
- T24 - Gates e pipelines independentes. Prioridade: P0. Responsabilidade: Cada serviço + Plataforma. Dependências: T11,T21. Critério: CI, Sonar bloqueante, contratos, manifests, publish/deploy por SHA ligados por dependência; falha bloqueia merge/deploy.
- T25 - Isolamento e cutover HML. Prioridade: P0. Responsabilidade: Dados + todos. Dependências: T07,T20,T22,T23,T24. Critério: Dados sintéticos, isolamento/rotas/grants e escritor único verificados; migração histórica é tarefa adicional somente se necessária.
- T26 - Traces e operação mínima. Prioridade: P1. Responsabilidade: Plataforma + todos. Dependências: T12 em diante, incremental. Critério: New Relic mostra os três serviços, etapas/compensação, pendências e procedimento de retry; sem central de alertas nova.
- T27 - Documentação e APIs de cada repo. Prioridade: P2. Responsabilidade: Cada serviço. Dependências: T04,T21,T22,T23,T24,T25,T26. Critério: README, Swagger ou Postman, schemas de mensagens, decisão de Saga, diagrama, cobertura e links; AsyncAPI formal é opcional.
- T28 - Ensaio final em Kubernetes. Prioridade: P1. Responsabilidade: Grupo. Dependências: T22-T27. Critério: Três serviços reais; fluxo completo, falha compensada, restart, deploy efetivo de um serviço, traces e integração MP teste.
- T29 - Gravar/revisar/publicar vídeo. Prioridade: P2. Responsabilidade: Grupo. Dependências: T28. Critério: YouTube/Vimeo público/não listado, <=15 min; demonstra todos os tópicos R34-R37 e acesso conferido.
- T30 - PDF e submissão no portal. Prioridade: P2. Responsabilidade: Grupo. Dependências: T27,T29. Critério: Participantes/IDs, URLs, vídeo, diagrama final, Saga e justificativas; PDF renderizado/revisado e checklist assinado.